



CS240 Algorithm Design and Analysis

Lecture 5

Dynamic Programming

Qingying Hao

Spring 2026

2026.03.17



Algorithmic Paradigms

- **Greed.** Build up a solution incrementally, myopically optimizing some local criterion
- **Divide-and-conquer.** Break up a problem into a few sub-problems, solve each sub-problem independently and recursively, and combine solution to sub-problems to form solution to original problem
- **Dynamic programming.** Break up a problem into a series of **overlapping** sub-problems, and build up solutions to larger and larger sub-problems

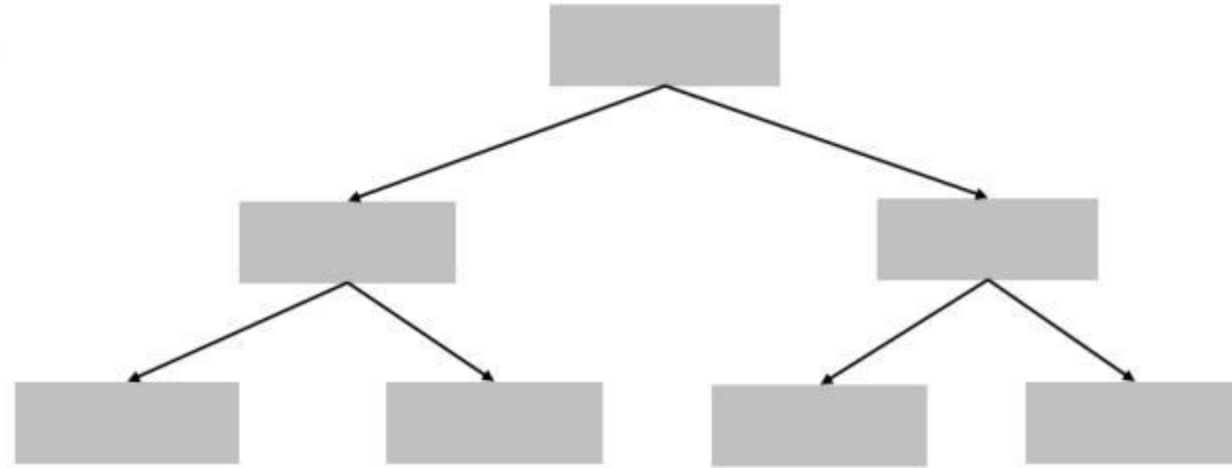




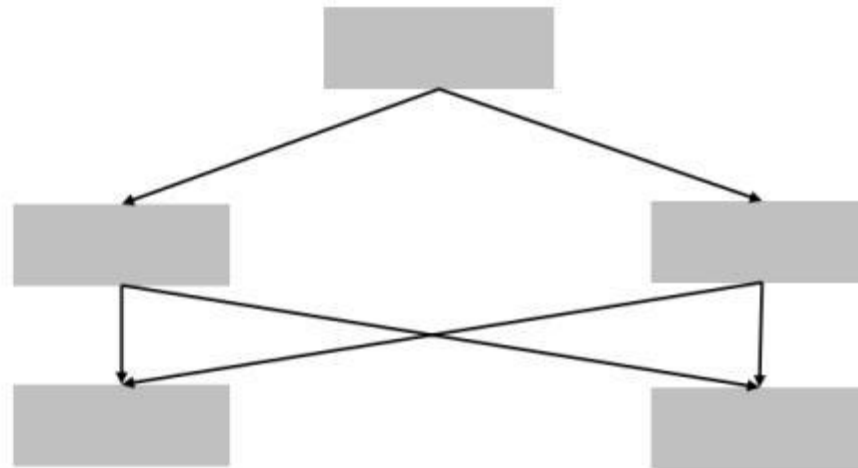
Divide-and-conquer VS. Dynamic Programming



Divide-and-conquer



Dynamic programming

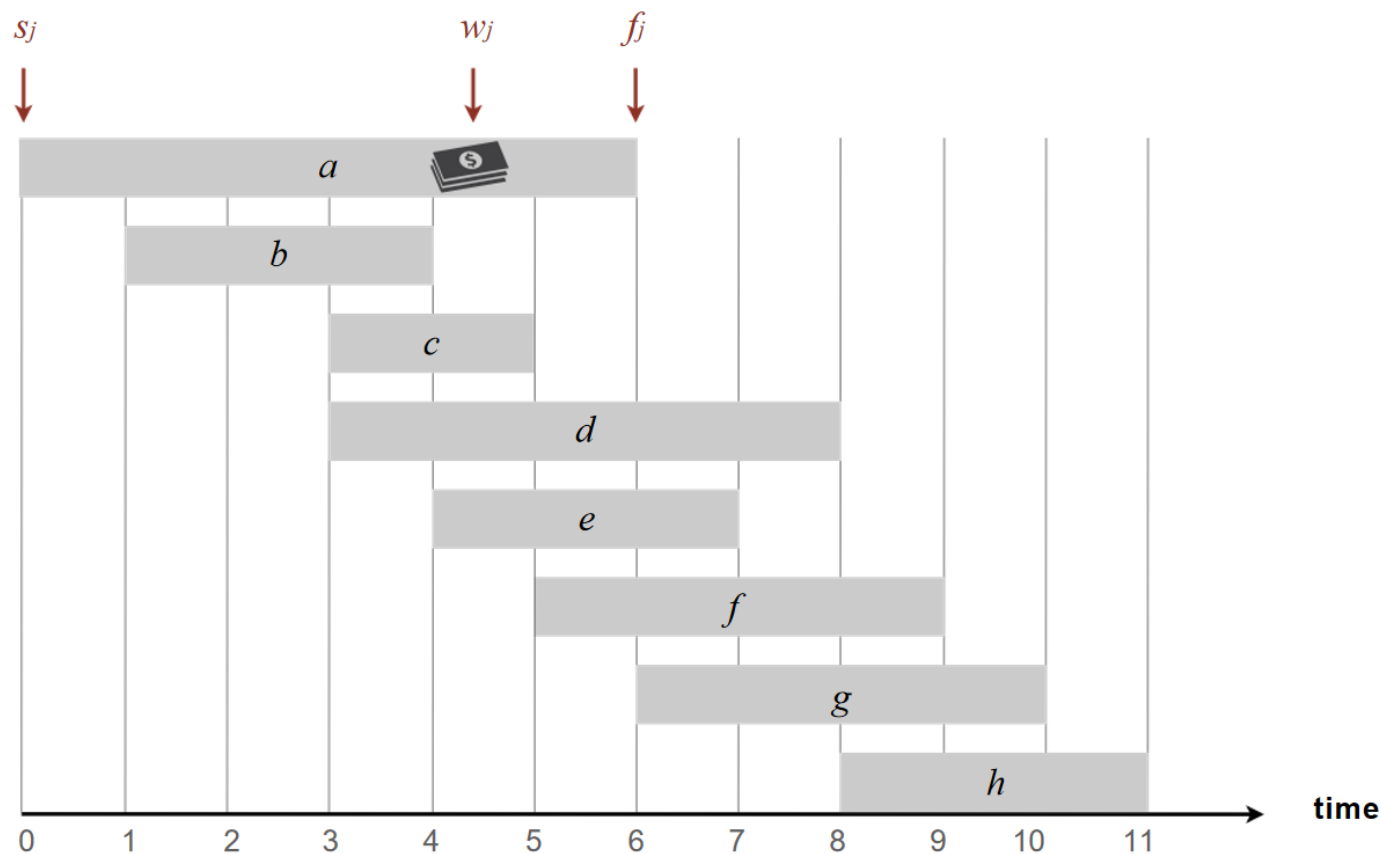




Weighted Interval Scheduling



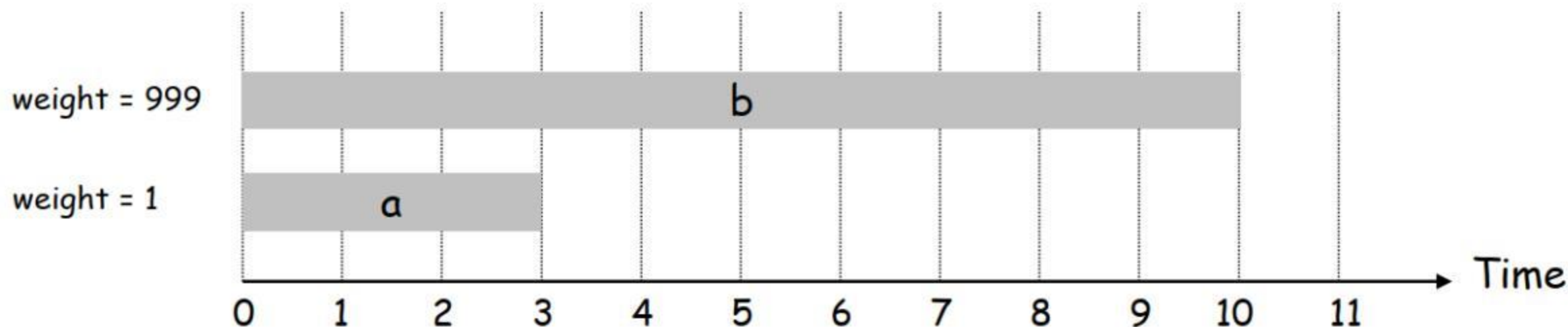
- Weighted interval scheduling problem
 - Job j starts at s_j , finishes at f_j , and has weight or value v_j
 - Two jobs **compatible** if they don't overlap
 - Goal: find maximum **weight** subset of mutually compatible jobs





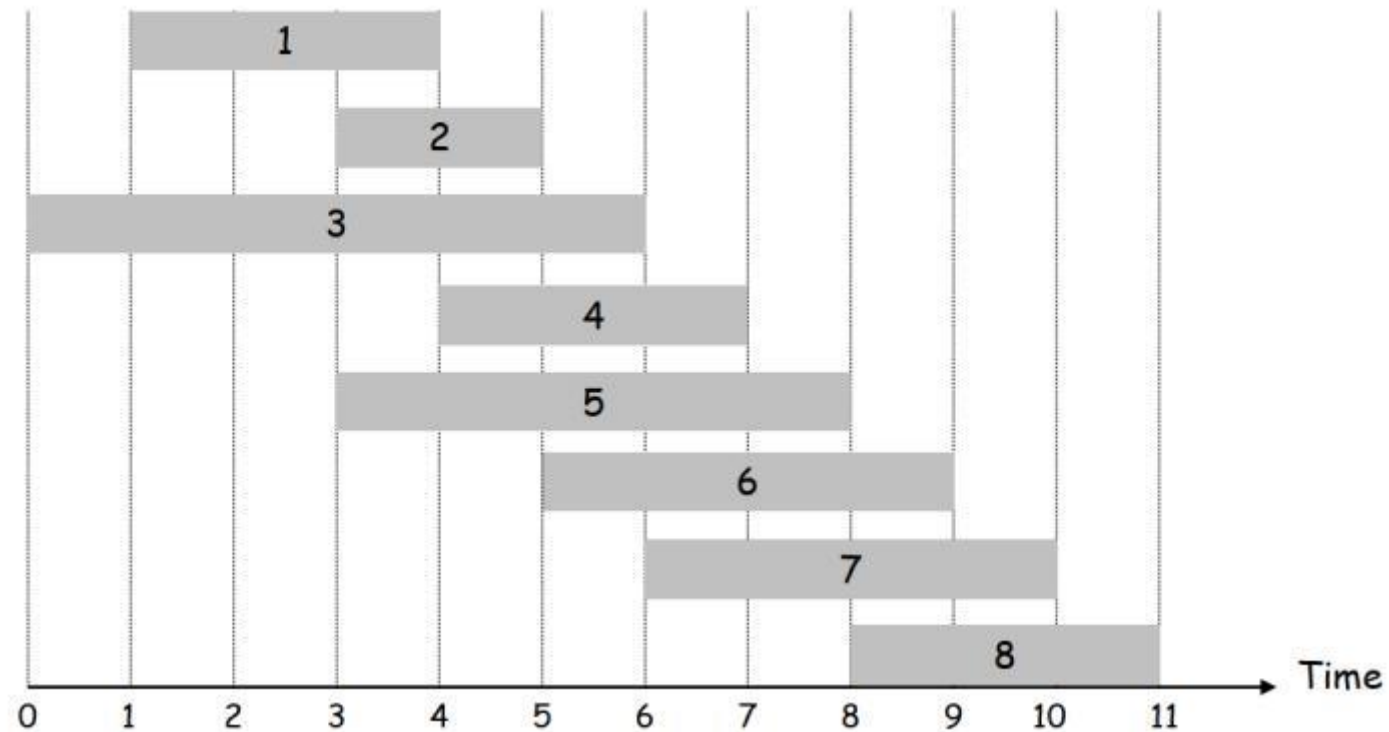
Unweighted Interval Scheduling Review

- **Recall.** Greedy algorithm works if all weights are 1
 - Consider jobs in ascending order of finish time
 - Add jobs to subset if it is compatible with previously chosen jobs
- **Observation.** Greedy algorithm can fail spectacularly if arbitrary weights are allowed



Weighted Interval Scheduling

- **Notation.** Label jobs by finishing time: $f_1 \leq f_2 \leq \dots \leq f_n$
- **Def.** $p(j)$ = largest index $i < j$ such that job i is compatible with j
i is rightmost interval that ends before j begins
- **Ex:** $p(8) = 5$, $p(7) = 3$, $p(2) = 0$





Dynamic Programming: Binary Choice

- **Notation.** $OPT(j)$ = value of optimal solution to the problem consisting of job requests 1, 2, ..., j
 - Case 1: OPT selects job j
 - Cannot use incompatible jobs $\{p(j) + 1, p(j) + 2, \dots, j-1\}$
 - Must include optimal solution to problem consisting of remaining compatible jobs 1, 2, ..., $p(j)$
 - Case 2: OPT does not select job j
 - Must include optimal solution to problem consisting remaining compatible jobs 1, 2, ..., $j-1$

←
← Optimal substructure

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max \{ v_j + OPT(p(j)), OPT(j-1) \} & \text{otherwise} \end{cases}$$

Case 1

Case 2





Weighted Interval Scheduling: Brute Force

- Brute force algorithm

Input: $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$

Sort jobs by finish times so that $f_1 \leq f_2 \leq \dots \leq f_n$.

Compute $p(1), p(2), \dots, p(n)$

Compute-Opt(j) {

if ($j = 0$)

return 0

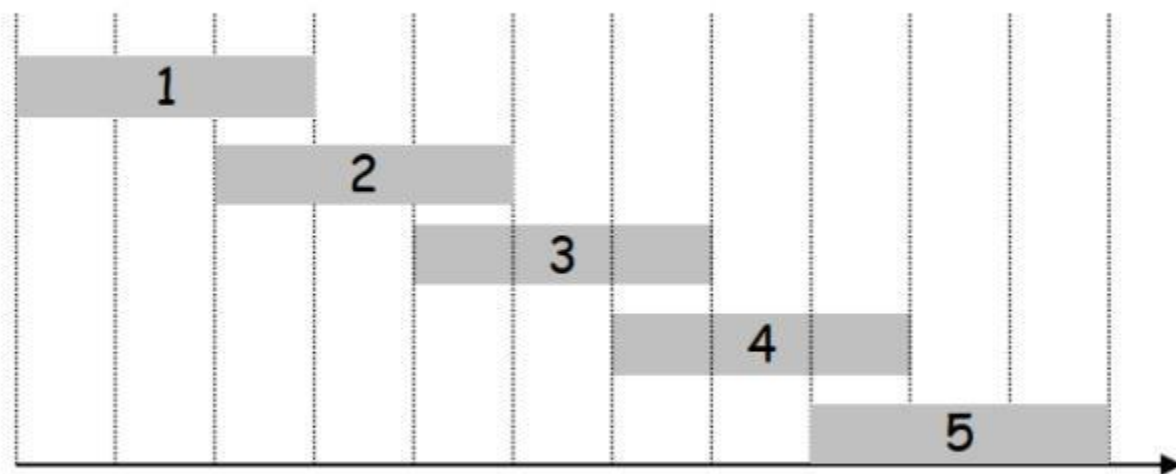
else

return $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j-1))$

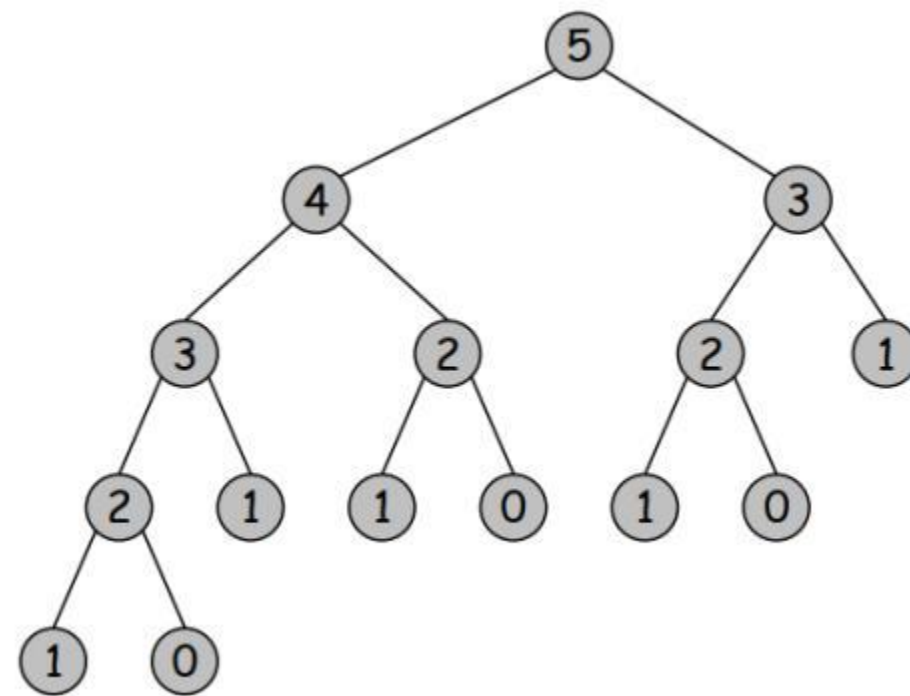
}

Weighted Interval Scheduling: Brute Force

- **Observation.** Recursive algorithm fails spectacularly because of redundant sub-problems → exponential algorithms
- **Ex.** Number of recursive calls for family of “layered” instances grows like Fibonacci sequence



$$p(1) = 0, p(j) = j-2$$



$$OPT(j) = \max\{v_j + OPT(j-2), OPT(j-1)\}$$

$$T(j) = T(j-1) + T(j-2) + O(1)$$



Weighted Interval Scheduling: Memoization

Top-down dynamic programming (memoization).

Cache result of subproblem j in $M[j]$

Use $M[j]$ to avoid solving subproblem j more than once.

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 
```

```
Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .
```

```
Compute  $p(1), p(2), \dots, p(n)$ 
```

```
for  $j = 1$  to  $n$ 
```

```
     $M[j] = \text{empty}$   $\leftarrow$  global array
```

```
 $M[0] = 0$ 
```

```
M-Compute-Opt( $j$ ) {
```

```
    if ( $M[j]$  is empty)
```

```
         $M[j] = \max(w_j + \text{M-Compute-Opt}(p(j)), \text{M-Compute-Opt}(j-1))$ 
```

```
    return  $M[j]$ 
```

```
}
```

Weighted Interval Scheduling: Running Time

- **Claim.** Memorized version of algorithm takes $O(n \log n)$ time
 - **Sort by finish time:** $O(n \log n)$
 - **Computing $p(\cdot)$:** $O(n)$ after sorting by start time ← how?
 - **M-Compute-Opt(j):** $O(n)$
 - Each entry $M[j]$ is computed only once. $M[j] = \text{OPT}(j)$
 - The computation of $M[j]$ invokes M-Compute-Opt twice

$$M[j] = \max (v_j + \text{M-Compute-Opt}(p(j)), \text{M-Compute-Opt}(j - 1))$$

- **Remark.** $O(n)$ if jobs are pre-sorted by start and finish times





Weighted Interval Scheduling: Running Time

- **Claim.** Memorized version of algorithm takes $O(n \log n)$ time
 - **Sort by finish time:** $O(n \log n)$
 - **Computing $p(\cdot)$:** $O(n)$ after sorting by start time ← how?

$$p(j) = \max\{i < j : f_i \leq s_j\}$$

Jobs sorted by finish time

Finish-sorted jobs:

index:	1	2	3	4	5	6
job:	1	2	3	4	5	6
finish:	2	4	5	7	9	10

The same jobs sorted by start time

Start-sorted jobs:

order:	1	2	3	4	5	6
job:	3	1	2	6	4	5
start:	0	1	3	5	6	8

Finish-sorted list:

job:	1	2	3	4	5	6
finish:	2	4	5	7	9	10

^

finish pointer

Start-sorted list:

job:	3	1	2	6	4	5
start:	0	1	3	5	6	8

^

current start job

$$p(3) = 0$$

Finish-sorted list:

job:	1	2	3	4	5	6
finish:	2	4	5	7	9	10

^

finish pointer

Start-sorted list:

job:	3	1	2	6	4	5
start:	0	1	3	5	6	8

^

current start job

$$p(1) = 0$$

$$\dots P(2) = 1$$





Weighted Interval Scheduling: Running Time

- **Claim.** Memorized version of algorithm takes $O(n \log n)$ time
 - **Sort by finish time:** $O(n \log n)$
 - **Computing $p(\cdot)$:** $O(n)$ after sorting by start time
 - the start pointer scans the start-sorted list once
 - the finish pointer scans the finish-sorted list once
 - neither pointer ever moves backward
 - **M-Compute-Opt(j):** $O(n)$
 - Each entry $M[j]$ is computed only once
 - The computation of $M[j]$ invokes M-Compute-Opt twice
- **Remark.** $O(n)$ if jobs are pre-sorted by start and finish times



Weighted Interval Scheduling: Finding a Solution

- **Q.** Dynamic programming algorithms computes optimal value. What if we want the solution itself?
- **A.** Do some post-processing

```
Run M-Compute-Opt(n)
Run Find-Solution(n)

Find-Solution(j) {
    if (j = 0)
        output nothing
    else if ( $v_j + M[p(j)] > M[j-1]$ )
        print j
        Find-Solution(p(j))
    else
        Find-Solution(j-1)
}
```

of recursive calls $\leq n \rightarrow O(n)$



Weighted Interval Scheduling: Bottom-Up

- **Bottom-up dynamic programming.** Unwind recursion

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 
```

```
Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .
```

```
Compute  $p(1), p(2), \dots, p(n)$ 
```

```
Iterative-Compute-Opt {
```

```
     $M[0] = 0$ 
```

```
    for  $j = 1$  to  $n$ 
```

```
         $M[j] = \max(v_j + M[p(j)], M[j-1])$ 
```

```
}
```

- **Top-down vs. bottom-up**
 - **Top-down:** May skip unnecessary sub-problems
 - **Bottom-up:** Save the overhead in recursion
 - **Running time:** The bottom-up version takes $O(n \log n)$ time





Knapsack Problem





Knapsack Problem



- **Knapsack problem**
 - Given n objects and a “knapsack”
 - Item i weighs $w_i > 0$ kilograms and has value $v_i > 0$
 - Knapsack has capacity of W kilograms
 - Goal: fill knapsack so as to maximize total value



10 oz., \$1,000



Max Weight: 400 oz.



100 oz., \$2,000



300 oz., \$4,000



1 oz., \$5,000



200 oz., \$5,000





Knapsack Problem

- **Knapsack problem**

- Given n objects and a “knapsack”
- Item i weighs $w_i > 0$ kilograms and has value $v_i > 0$
- Knapsack has capacity of W kilograms
- Goal: fill knapsack so as to maximize total value

- **Ex:** { 3, 4 } has value 40

$W = 11$

- **Greedy:**

- Repeatedly add item with maximum value v_i
- Repeatedly add item with maximum weight w_i
- Repeatedly add item with maximum ration v_i/w_i

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Greedy not optimal!





Dynamic Programming: False Start



- **Def.** $\text{OPT}(i) = \max$ profit subset of items $1, \dots, i$
 - Case 1: OPT does not select item i
 - OPT selects best of $\{ 1, 2, \dots, i-1 \}$
 - Case 2: OPT selects item i
 - How shall we enforce the weight limit?
- **Conclusion.** Shall specify the remaining weight capacity in OPT





Dynamic Programming: Adding a New Variable



- **Def.** $OPT(i, w)$ = max profit subset of items $1, \dots, i$ with weight limit w
 - Case 1: OPT does not select item i
 - OPT selects best of $\{ 1, 2, \dots, i-1 \}$ using weight limit w
 - Case 2: OPT selects item i
 - New weight limit = $w - w_i$
 - OPT selects best of $\{ 1, 2, \dots, i-1 \}$ using this new weight limit

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$





Knapsack Problem: Bottom-Up



- **Knapsack.** Fill up an n -by- W array

```
Input:  $n, w_1, \dots, w_N, v_1, \dots, v_N$ 

for  $w = 0$  to  $W$ 
     $M[0, w] = 0$ 

for  $i = 1$  to  $n$ 
    for  $w = 1$  to  $W$ 
        if  $(w_i > w)$ 
             $M[i, w] = M[i-1, w]$ 
        else
             $M[i, w] = \max \{M[i-1, w], v_i + M[i-1, w-w_i]\}$ 

return  $M[n, W]$ 
```





Knapsack Algorithm

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

W=11

subset
of items
1, ..., i

	weight limit w											
	0	1	2	3	4	5	6	7	8	9	10	11
{ }	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
{ 1, 2, 3, 4 }	0	1	6	7	7	18	22	24	28	29	29	40
{ 1, 2, 3, 4, 5 }	0	1	6	7	7	18	22	28	29	34	35	40

OPT(i, w) = optimal value of knapsack problem with items 1, ..., i, subject to weight limit w

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

subset
of items
1, ..., i

		weight limit w											
		0	1	2	3	4	5	6	7	8	9	10	11
	{ }	0	0	0	0	0	0	0	0	0	0	0	0
	{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
	{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
	{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
		1	6	7	7	18	22	24	28	29	29	29	40
		1	6	7	7	18	22	28	29	34	35	40	40

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max\{OPT(i-1, w), v_i + OPT(i-1, w-w_i)\} & \text{otherwise} \end{cases}$$

$OPT(i, w)$ = optimal value of knapsack problem with items 1, ..., i, subject to weight limit w

First row : if $i = 0$ $OPT(i, w) = 0$ if $i = 0$

Second row{1}: if $w = 0$ $OPT(i, w) = 0$; if $w = 1$, $OPT = 1$

Third row {1,2}: item 1(1,1) item2: (2,6)

If $w = 1$, weight for item 2 is $2 > 0$, $OPT(2,1)=OPT(1,1)=1$

If $w = 2$, $OPT(2,2) = \max\{\text{opt}(1,2), 6+\text{opt}(1,0)\} = 6$

If $w = 3$, $OPT(2,3) = \max\{\text{opt}(1,3), 6+\text{opt}(1,1)\} = 7$

Fourth row: item 1(1,1) item2: (2,6) item3: (5, 18)

If $w = 5$, $\text{opt}(3,5) = \max\{\text{opt}(2,5), 18+\text{opt}(2,0)\} = 18$

If $w = 6$, $\text{opt}(3,6) = \max\{\text{opt}(2,6), 18+\text{opt}(3,1)\} = 19 \dots w7 = 24$



Knapsack Problem: Running Time



Theorem. The DP algorithm solves the knapsack problem with n items and maximum weight W in $\Theta(n W)$ time and $\Theta(n W)$ space.

Pf.

← weights are integers
between 1 and W

Takes $O(1)$ time per table entry.

There are $\Theta(n W)$ table entries.

After computing optimal values, can trace back to find solution:

$OPT(i, w)$ takes item i iff $M[i, w] > M[i - 1, w]$.

Remarks.

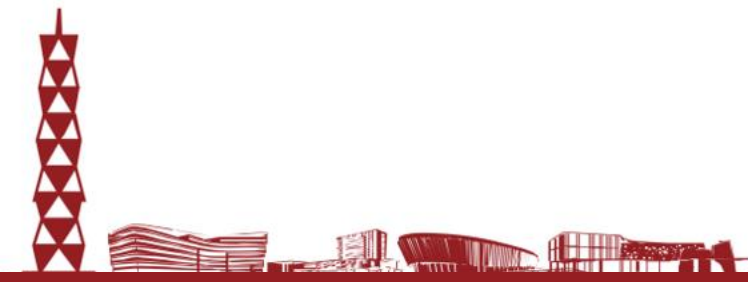
Algorithm depends critically on assumption that weights are integral.

Assumption that values are integral was not used.





RNA Secondary Structure

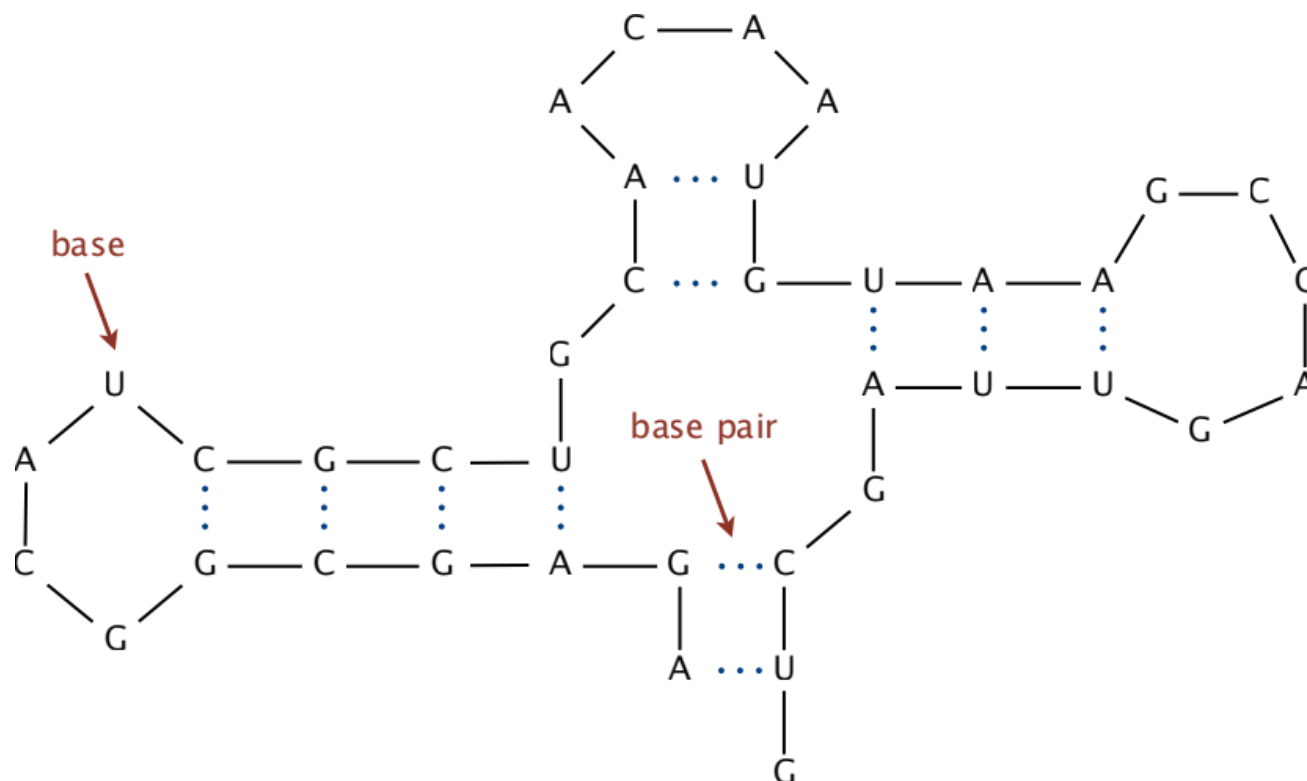




RNA Secondary Structure



- **RNA.** String $B = b_1 b_2 \dots b_n$ over alphabet $\{A, C, G, U\}$
- **Secondary structure.** RNA is single-stranded so it tends to loop back and form base pairs with itself. This structure is essential for understanding behavior of molecule.



RNA secondary structure for GUCGAUUGAGCGAAUGUAACAACGUGGCUACGGCGAGA





RNA Secondary Structure

- **Secondary structure.** A set of pairs $S = \{ (b_i, b_j) \}$ that satisfy:
 - [Watson-Crick]. S is matching and each pair in S is a Watson-Crick complement: A-U, U-A, C-G, or G-C
 - [No sharp turns]. The ends of each pair are separated by at least 4 intervening bases. If $(b_i, b_j) \in S$, then $i < j - 4$
 - [Non-crossing]. If (b_i, b_j) and (b_k, b_l) are two pairs in S , then we cannot have $i < k < j < l$

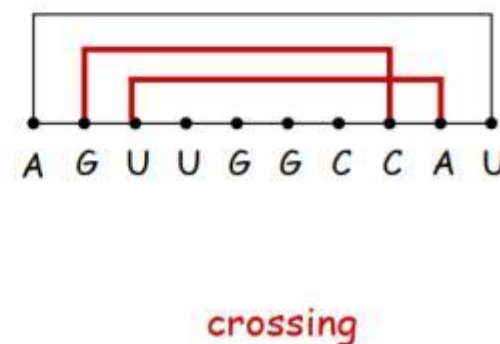
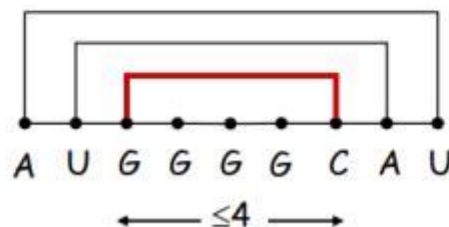
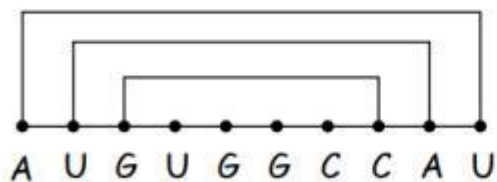
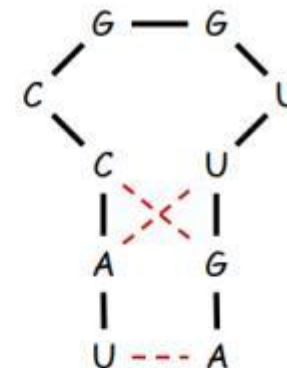
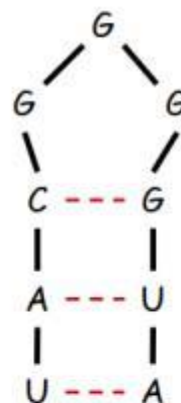
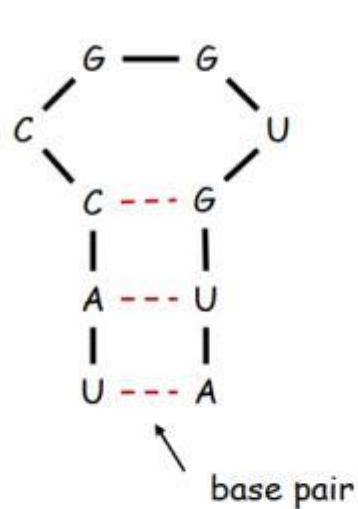




RNA Secondary Structure: Examples



- Examples





RNA Secondary Structure



- **Free energy.** Usual hypothesis is that an RNA molecule will form the secondary structure with the optimum total free energy



approximate by number of base pairs

- **Goal.** Given an RNA molecule $B = b_1 b_2 \dots b_n$ find a secondary structure S that maximizes the number of base pairs

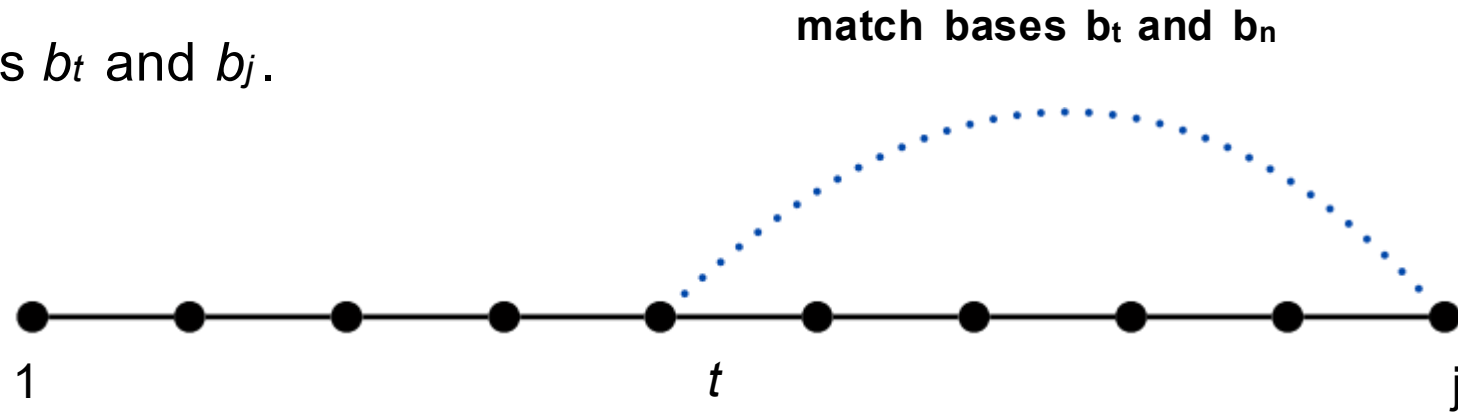


RNA Secondary Structure: Subproblems

First attempt. $OPT(j)$ = maximum number of base pairs in a secondary structure of the substring $b_1b_2 \dots b_j$.

Goal. $OPT(n)$.

Choice. Match bases b_t and b_j .



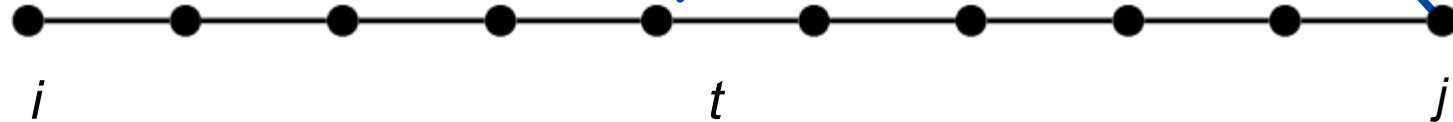
Difficulty. Results in two subproblems (but one of wrong form).

- Find secondary structure in $b_1b_2 \dots b_{t-1}$. ← $OPT(t-1)$
- Find secondary structure in $b_{t+1}b_{t+2} \dots b_{j-1}$. ← need more subproblems (first base no longer b_1)



Dynamic Programming Over Intervals

- **Notation.** $\text{OPT}(i, j)$ = maximum number of base pairs in a secondary structure of the substring $b_i b_{i+1} \dots b_j$
- If $i \geq j - 4$
 - $\text{OPT}(i, j) = 0$ by no-sharp turns condition
- If $i < j - 4$: take max of two cases
 - Case 1. Base b_j is not involved in a pair
 - $\text{OPT}(i, j - 1)$
 - Case 2. **Base b_j pairs with b_t** for some $i \leq t < j - 4$ $b_i \dots b_{t-1}$, $b_{t+1} \dots b_{j-1}$
Non-crossing constraint decouples resulting sub-problems
 $1 + \max_t \{ \text{OPT}(i, t-1) + \text{OPT}(t+1, j-1) \}$



take max over t such that $i \leq t < j - 4$ and b_t and b_j are Watson-Crick complements

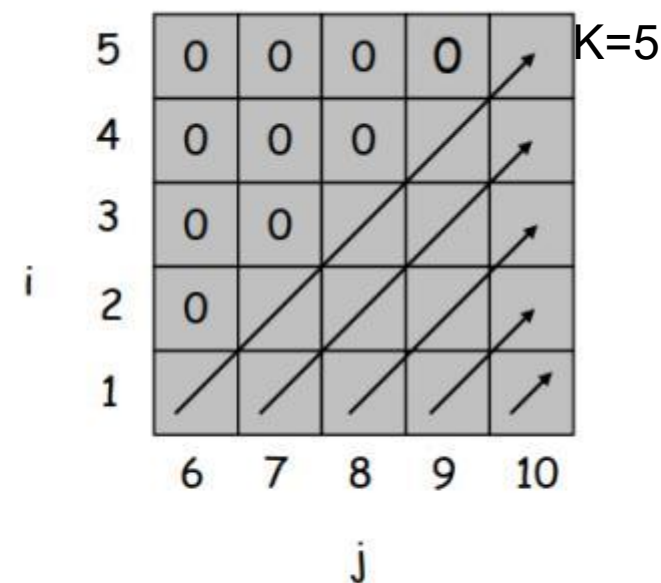




Bottom Up Dynamic Programming Over Intervals

- **Q.** What order to solve the sub-problems?
- **A.** Do shortest intervals first
 $K = j - i$

```
RNA( $b_1, \dots, b_n$ ) {  
    for  $k = 5, 6, \dots, n-1$   
        for  $i = 1, 2, \dots, n-k$   
             $j = i + k$   
            Compute  $M[i, j]$   
  
    return  $M[1, n]$   
}
```



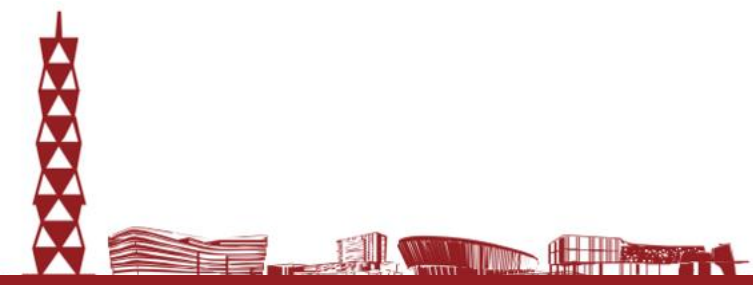
$$M[i, j] = \max \left(M[i, j-1], \max_t (1 + M[i, t-1] + M[t+1, j-1]) \right)$$

Theorem. The DP algorithm solves the RNA secondary structure problem in $O(n^3)$ time and $O(n^2)$ space. Interval DP + enumeration of split point





Sequence Alignment



String Similarity

- How similar are two strings?
 - occurrence
 - occurrence

o c u r r a n c e -

o c c u r r e n c e

6 mismatches, 1 gap

o c - u r r a n c e

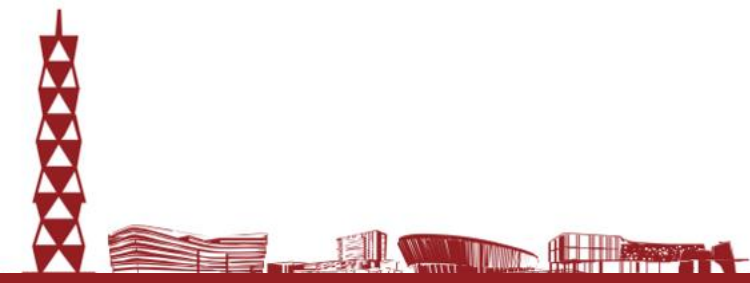
o c c u r r e n c e

1 mismatch, 1 gap

o c - u r r - a n c e

o c c u r r e - n c e

0 mismatches, 3 gaps



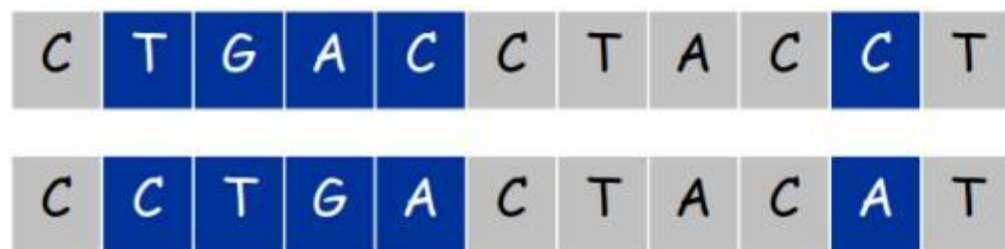


Edit Distance

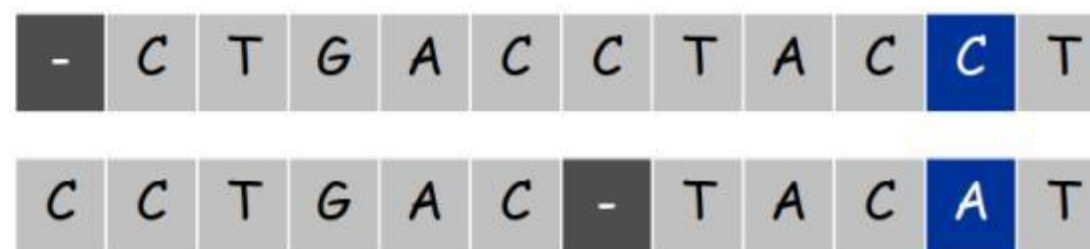


- **Edit distance.** [Levenshtein 1966, Needleman-Wunsch 1970]

- Gap penalty δ ; mismatch penalty α_{pq}
- Cost = sum of gap and mismatch penalties
- Edit distance = min cost



$$\alpha_{TC} + \alpha_{GT} + \alpha_{AG} + 2\alpha_{CA}$$



$$2\delta + \alpha_{CA}$$

- **Applications**

- Basis for Unix diff
- Speech recognition
- Computational biology





Sequence Alignment

- **Goal.** Given two strings $X = x_1x_2\dots x_m$ and $Y = y_1y_2\dots y_n$ find alignment of minimum cost
- **Def.** An **alignment** M is a set of ordered pairs $x_i - y_j$ such that each item occurs in at most one pair and no crossings
 - The pair $x_i - y_j$ and $x_{i'} - y_{j'}$ cross if $i < i'$, but $j > j'$ $i < i' \Rightarrow j < j'$
- **Def.** The cost of an alignment

$$\text{cost}(M) = \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\sum_{i: x_i \text{ unmatched}} \delta + \sum_{j: y_j \text{ unmatched}} \delta}_{\text{gap}}$$

- **Ex:** An alignment of CTACCG vs. TACATG

$M = x_2 - y_1, x_3 - y_2, x_4 - y_3, x_5 - y_4, x_6 - y_6$

x_1	x_2	x_3	x_4	x_5		x_6
C	T	A	C	C	-	G
-	T	A	C	A	T	G
	y_1	y_2	y_3	y_4	y_5	y_6



Sequence Alignment: Problem Structure



- **Def.** $OPT(i, j)$ = min cost of aligning strings $x_1x_2\dots x_i$ and $y_1y_2\dots y_j$
- Case 1: OPT align x_i with y_j $x_i \ y_j$
 - pay **mismatch** for $x_i - y_j$ + min cost of aligning two strings $x_1x_2\dots x_{i-1}$ and $y_1y_2\dots y_{j-1}$
- Case 2a: OPT leaves x_i unmatched $x_i \ -$
 - pay **gap** for x_i and min cost of aligning $x_1x_2\dots x_{i-1}$ and $y_1y_2\dots y_j$
- Case 2b: OPT leaves y_j unmatched
 - pay gap for y_j and min cost of aligning $x_1x_2\dots x_i$ and $y_1y_2\dots y_{j-1}$ $- \ y_j$

$$OPT(i, j) = \begin{cases} j\delta & \text{if } i = 0 \\ \min \begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \\ i\delta & \text{if } j = 0 \end{cases}$$

$i = 0$:
X: - - - ... -
Y: $y_1 \ y_2 \ \dots \ y_j$
cost = $j \cdot \delta$

$j = 0$:
X: $x_1 \ x_2 \ \dots \ x_i$
Y: - - - ... -
cost = $i \cdot \delta$





Sequence Alignment: Algorithm



```
Sequence-Alignment( $m, n, x_1x_2\ldots x_m, y_1y_2\ldots y_n, \delta, \alpha$ ) {  
  for  $i = 0$  to  $m$   
    M[ $i, 0$ ] =  $i\delta$   
  for  $j = 0$  to  $n$   
    M[ $0, j$ ] =  $j\delta$   
  
  for  $i = 1$  to  $m$   
    for  $j = 1$  to  $n$   
      M[ $i, j$ ] = min( $\alpha[x_i, y_j] + M[i-1, j-1],$   
                     $\delta + M[i-1, j],$   
                     $\delta + M[i, j-1]$ )  
  
  return M[ $m, n$ ]  
}
```

X: $x_1 x_2 \dots x_i$

Y: -----

X: -----

Y: $y_1 y_2 \dots y_j$

Analysis: $\Theta(mn)$ time and space





Sequence Alignment: Example



		P	A	L	A	T	E	
		0	2	4	6	8	10	12
P		2	0	2	4	6	8	10
A		4	2	0	2	4	6	8
L		6	4	2	0	2	4	6
E		8	6	4	2	1	3	4
T		10	8	6	4	3	1	3
T		12	10	8	6	5	3	2
E		14	12	10	8	7	5	3

PAL
P--
 $\text{OPT}(3, 1) = 4$

P	A	L	E	T	T	E
P	A	L	-	A	T	E

1 mismatch, 1 gap

Match: 0

Mismatch: 1

Gap: 2

↖ means align x_i with y_j

↑ means align x_i with a gap

← means align y_j with a gap



Sequence Alignment: Algorithm



- **Analysis:** $\Theta(mn)$ time and space
- **English words or sentences:**
 - $m, n \leq 30$. $\leftarrow$ OK
- **Computational biology**
 - $m = n = 100,000$
 - 10 billions ops is OK, but 10GB array is quite large



Sequence Alignment: Linear Space

- **Q.** Can we avoid using quadratic **space**?
- **Easy.** Optimal **cost** in $O(m + n)$ space and $O(mn)$ time
 - Compute $\text{OPT}(i, .)$ from $\text{OPT}(i-1, .)$
 - No longer a simple way to recover alignment itself
- **Theorem.** [Hirschberg 1975] Optimal **alignment** in $O(m + n)$ space and $O(mn)$ time
 - Clever combination of divide-and-conquer and dynamic programming

$\text{OPT}(i, j)$ depends on :

- $\text{OPT}(i-1, j-1)$

- $\text{OPT}(i-1, j)$

- $\text{OPT}(i, j-1)$

prev_row[0..n]

curr_row[0..n]

$O(m + n)$





Sequence Alignment in Linear Space



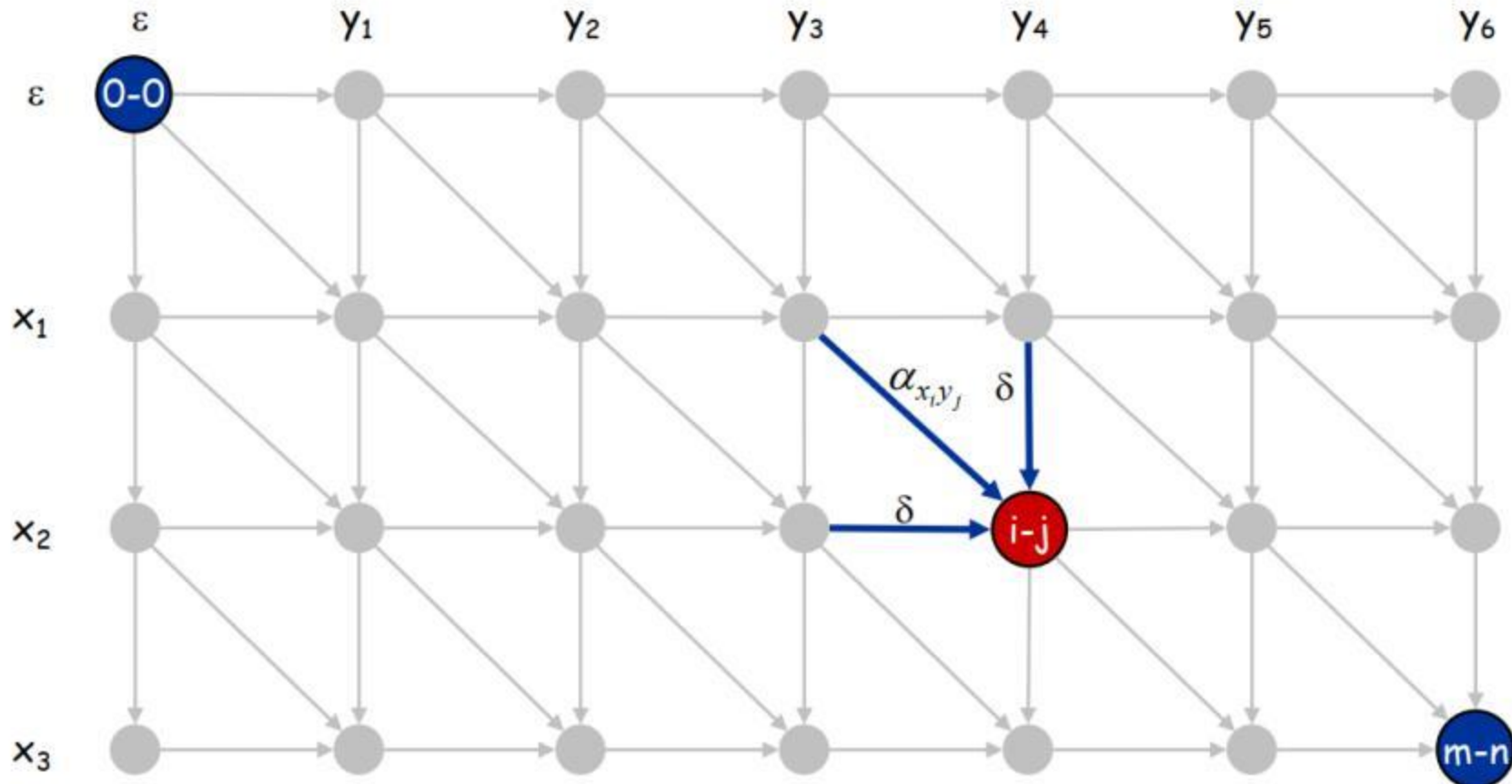


Sequence Alignment: Linear Space



- **Edit distance graph**

- Let $f(i, j)$ be shortest path from $(0, 0)$ to (i, j)
- Observation: $f(i, j) = \text{OPT}(i, j)$



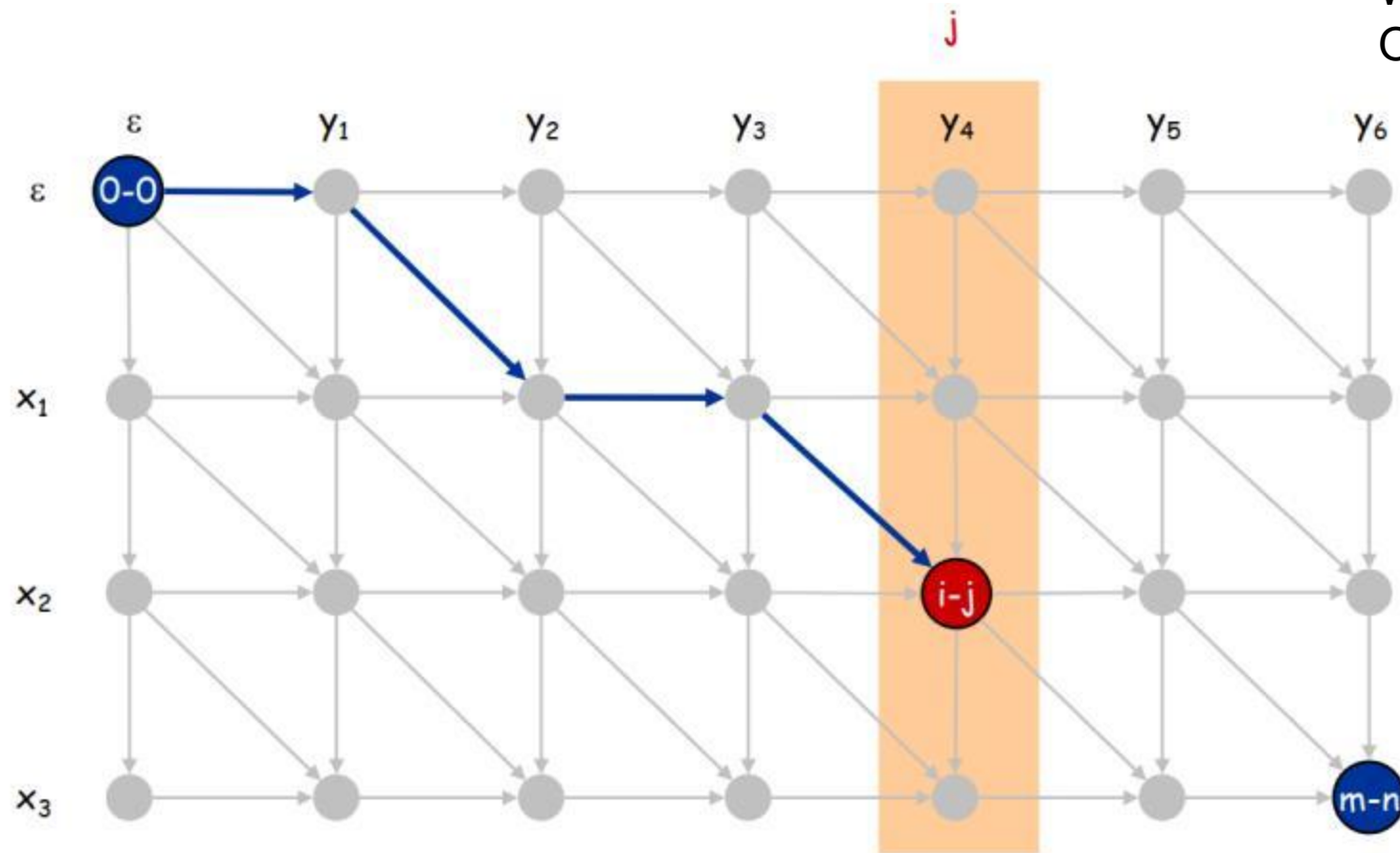


Sequence Alignment: Linear Space

- **Edit distance graph**

- Let $f(i, j)$ be shortest path from $(0, 0)$ to (i, j)
- Can compute $f(., j)$ for any j in $O(mn)$ time and $O(m + n)$ space

$f(i-1, j)$
 $f(i, j-1)$
 $f(i-1, j-1)$
When calculating j :
Only depend on $j-1$



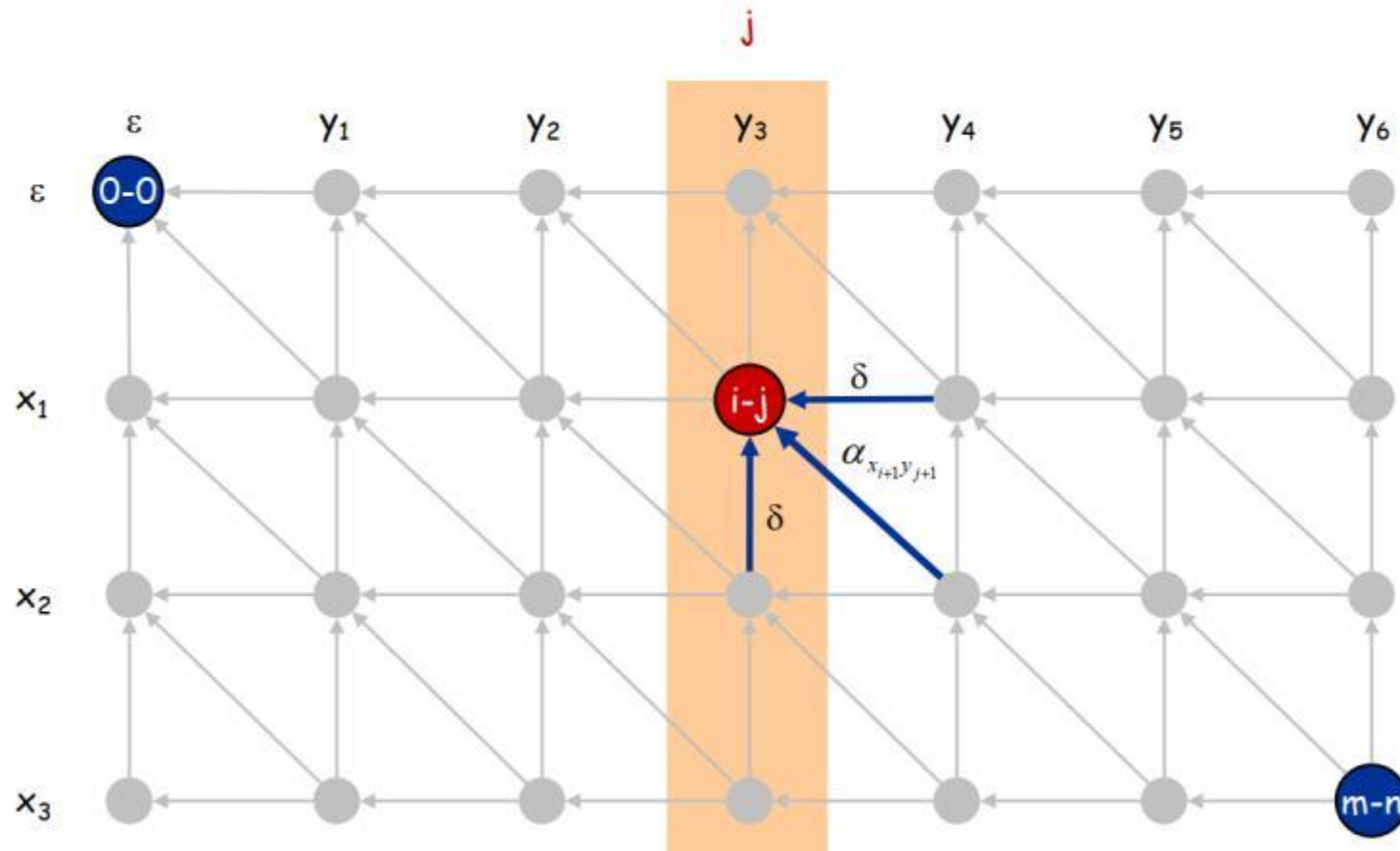


Sequence Alignment: Linear Space



- **Edit distance graph**

- Let $g(i, j)$ be shortest path from (i, j) to (m, n)
- Can compute $g(., j)$ by reversing the edge orientations and inverting the roles of $(0, 0)$ and (m, n)



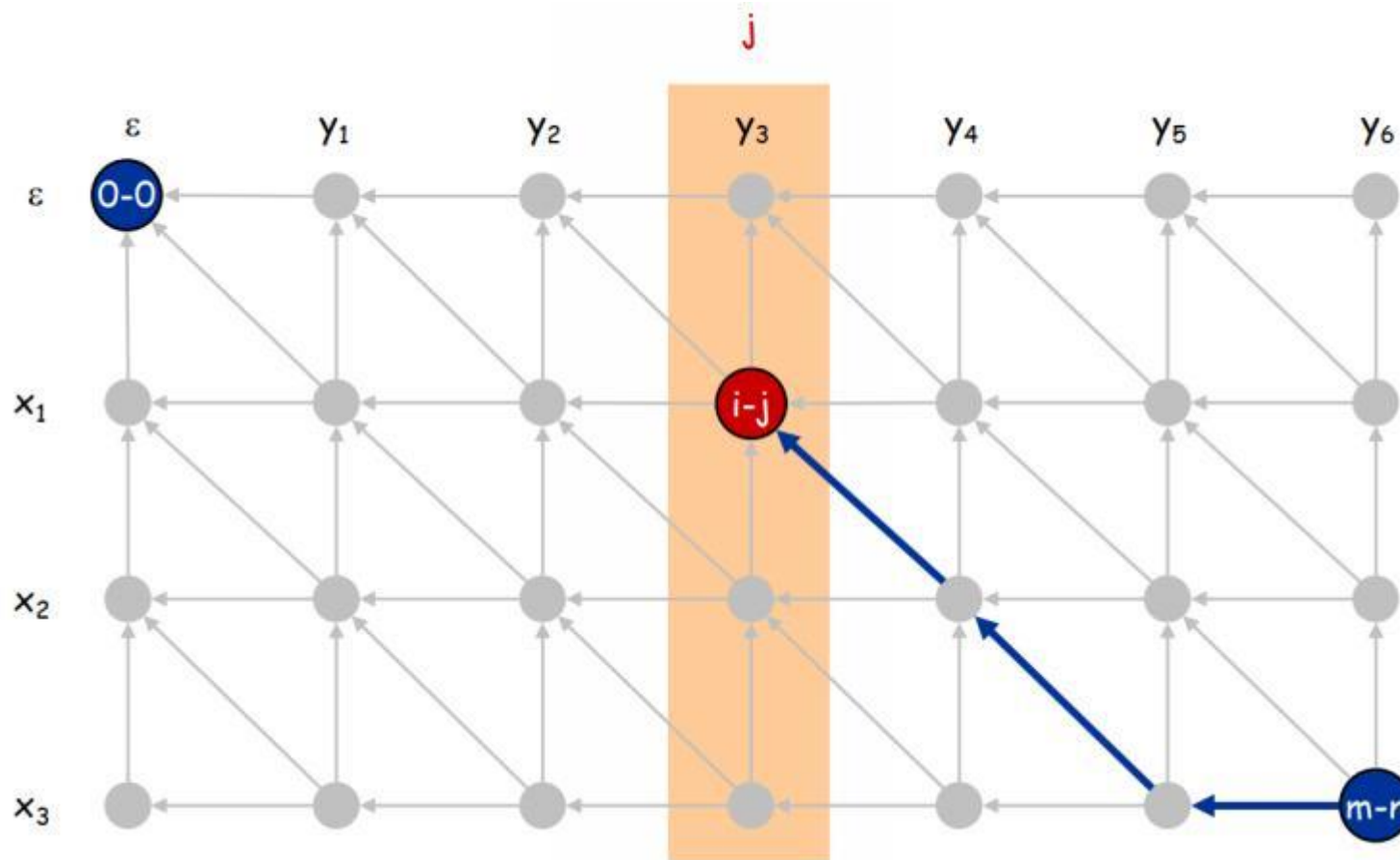


Sequence Alignment: Linear Space



- **Edit distance graph**

- Let $g(i, j)$ be shortest path from (i, j) to (m, n)
- Can compute $g(\cdot, j)$ for any j in $O(mn)$ time and $O(m + n)$ space

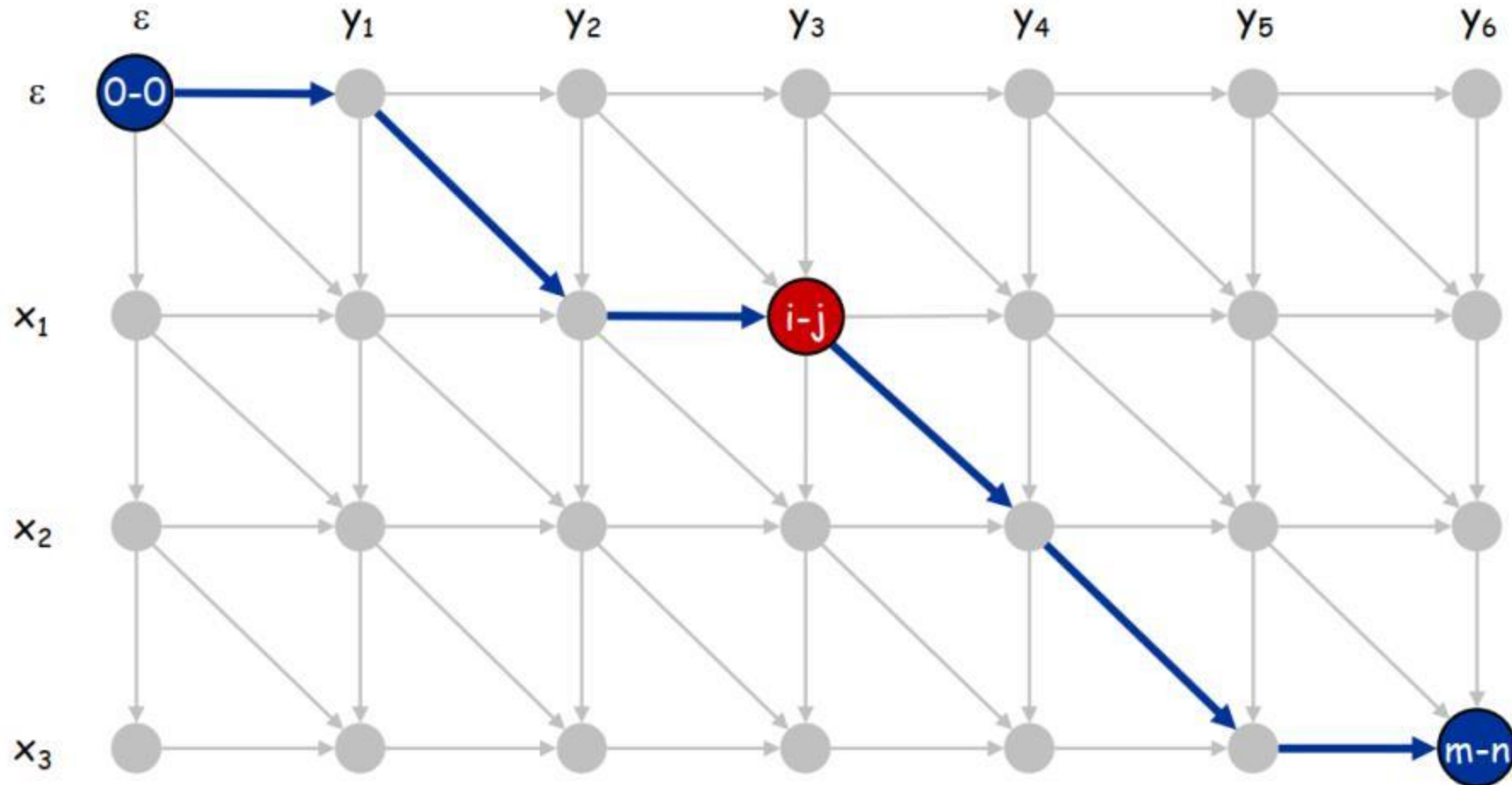




Sequence Alignment: Linear Space



- **Observation 1.** The cost of the shortest path that uses (i, j) is $f(i, j) + g(i, j)$

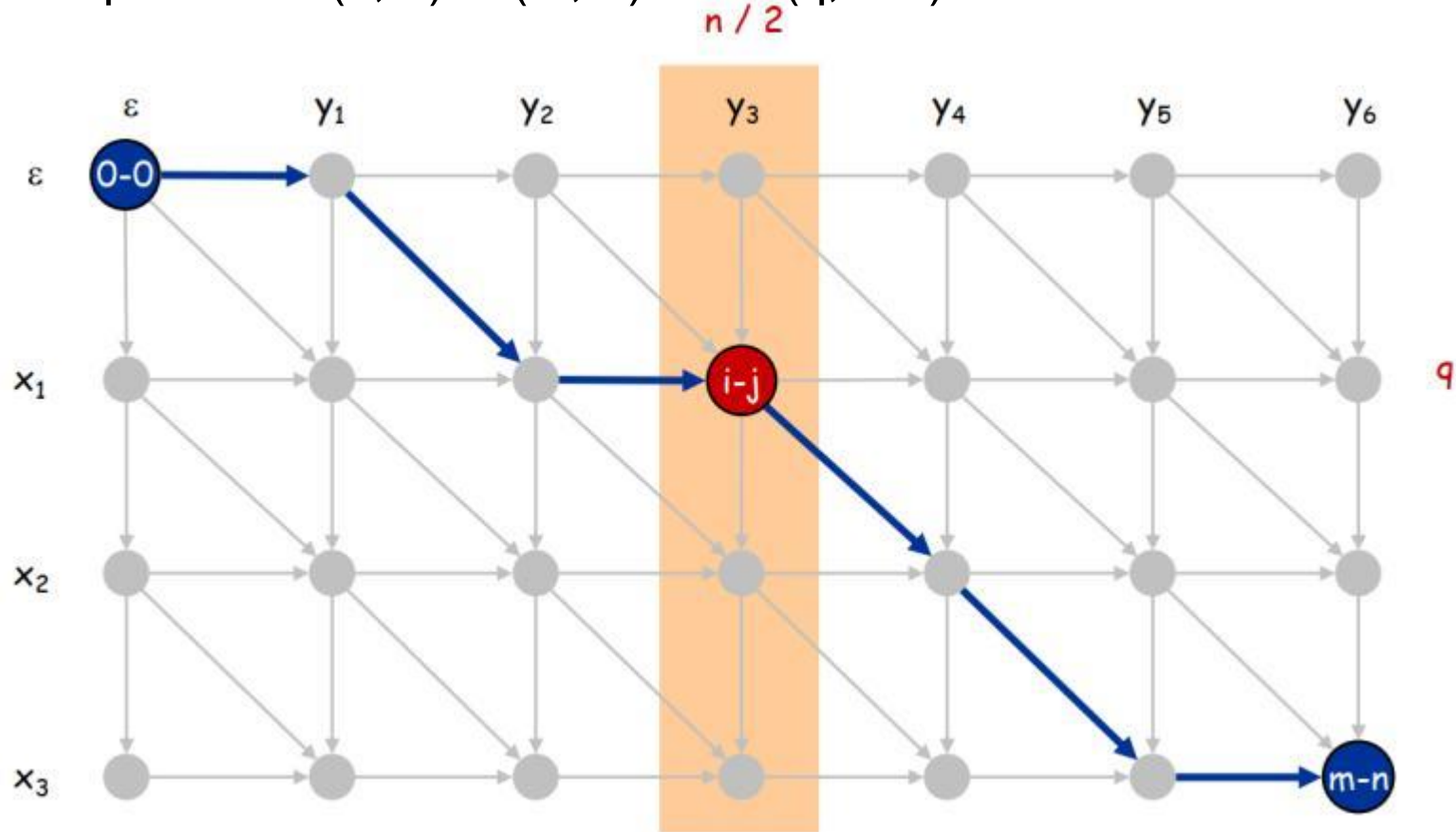




Sequence Alignment: Linear Space

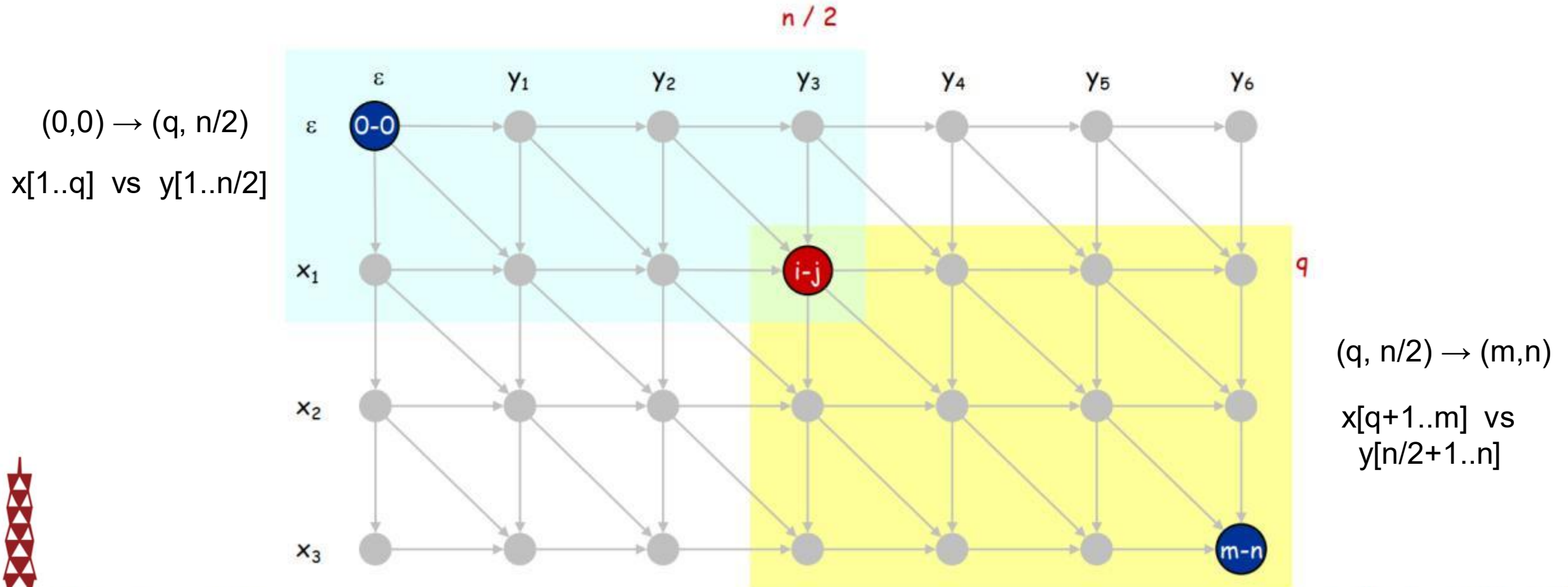


- **Observation 2.** Let q be an index that minimized $f(q, n/2) + g(q, n/2)$
 $q = \operatorname{argmin}_i [f(i, n/2) + g(i, n/2)]$
- Then, the shortest path from $(0, 0)$ to (m, n) uses $(q, n/2)$



Sequence Alignment: Linear Space

- Divide: find index q that minimizes $f(q, n/2) + g(q, n/2)$ using DP
- Do alignment at $(x_q, y_{n/2})$
- Conquer: recursively compute optimal alignment in each piece





Sequence Alignment: Running Time Analysis

- **Theorem.** Let $T(m, n)$ = **max** running time of algorithm on strings of length m and n .

$$T(m, n) = O(mn)$$

- **Pf.** (by induction on n)
 - $O(mn)$ time to compute $f(., n/2)$ and $g(., n/2)$ and find index q
 - $T(q, n/2) + T(m - q, n/2)$ time for two recursive calls (left, right)
 - Choose constant c so that:

$$T(m, 2) \leq cm$$

$$T(2, n) \leq cn$$

$$T(m, n) \leq cmn + T(q, n/2) + T(m - q, n/2)$$

- Claim: $T(m, n) \leq 2cmn$
 - Base cases: $m = 2$ or $n = 2$
 - Inductive hypothesis: $T(m', n') \leq 2cm'n'$ with $m' < m$ and $n' < n$

$$\begin{aligned} T(m, n) &\leq T(q, n/2) + T(m - q, n/2) + cmn \\ &\leq 2cq(n/2) + 2c(m - q)(n/2) + cmn \\ &= cq(n/2) + c(m - q)(n/2) + cmn \\ &= 2cmn \end{aligned}$$

$$T(q, n/2) \leq 2cq(n/2)$$

$$T(m - q, n/2) \leq 2c(m - q)(n/2)$$



Next Time: Dynamic Programming (Cont.)

